SRINIVAS UNIVERSITY

INSTITUTE OF ENGINEERING & TECHNOLOGY
MUKKA,MANGALURU - 574146

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING ENGINEERING

FUNDAMENTALS OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING
GROUP TASK REPORT
ON

" PRACTICAL APPLICATIONS OF BIG DATA, MACHINE LEARNING, NEURAL NETWORKS, AND GENERATIVE AI "

Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY
IN
ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING ENGINEERING

Submitted By,

GAYATHRI	**01SU25AI030**
JENISHA JILSON	**01SU25AI032**
JASIR SULTHAN	**01SU25AI031**
ASHIN K P	**01SU25AI017**

UNDER THE GUIDANCE OF

PROF. MAHESH KUMAR VB

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING ENGINEERING

2025-2026

SRINIVAS UNIVERSITY

INSTITUTE OF ENGINEERING & TECHNOLOGY
MUKKA,MANGALURU - 574146

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING ENGINEERING

CERTIFICATE

This is to certify that the project entitled"PRACTICAL APPLICATIONS OF BIG DATA, MACHINE LEARNING, NEURAL NETWORKS, AND GENERATIVE AI " is a bona fide work carried out by

GAYATHRI	**01SU25AI030**
JENISHA JILSON	**01SU25AI032**
JASIR SULTHAN	**01SU25AI031**
ASHIN K P	**01SU25AI017**

in the partial fulfillment for the award of BACHELOR OF TECHNOLOGY (B.Tech) of the Srinivas University Institute of Engineering and Technology during the year 2025-2026. It is certified that all corrections/suggestions indicated for internal assessment have been incorporated in the report deposited in the department library. The project report has been approved as it satisfies the academic requirements in respect of project work prescribed for the Bachelor of Technology Degree.

PROF. MAHESH KUMAR VB
Professor, Dept. AIML

SRINIVAS UNIVERSITY

INSTITUTE OF ENGINEERING & TECHNOLOGY
MUKKA,MANGALURU - 574146

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING ENGINEERING

DECLARATION

We, GAYATHRI - 01SU25AI030, JENISHAJILSON-01SU25AI032, JASIR SULTHAN -01SU25AI031, ASHIN K P - 01SU25AI017, students of BACHELOR OF TECHNOLOGY, Srinivas University, Mukka, hereby declare that the group task entitled " PRACTICAL APPLICATIONS OF BIG DATA, MACHINE LEARNING, NEURAL NETWORKS, AND GENERATIVE AI " has been successfully completed by us in partial fulfillment of the requirements for the award of degree in BACHELOR OF TECHNOLOGY, Srinivas University Institute of Engineering and Technology, and no part of it has been submitted for the award of any other degree or diploma.

Date: 12-10-2026

Place : Mukka

CONTENTS

Module 2 BIG DATA PROCESS MAPPING	**2**
2.1 Introduction and Problem Statement	2
2.2 Data Sources	2
2.3 Data Ingestion and Storage	3
2.4 Processing Layer	3
2.5 Output Layer and Feedback Loop	3
2.6 The Five Vs Applied to Google Maps	4
Module 3 BUILD A SIMPLE ML PROCESS FLOW	**4**
3.1 Introduction and Problem Definition	4
3.2 Data Collection and Preparation	4
3.3 Feature Extraction and Algorithm Selection	5
3.4 Training and Validation	6
3.5 Testing, Evaluation and Deployment	6
Module 4 BACKPROPAGATION WALKTHROUGH	**7**
4.1 Introduction and Network Setup	7
4.2 Forward Pass and Loss	7
4.3 Backward Pass and Gradient Flow	7
4.4 Weight Update and Verification 4.5	8
Observations	8
Module 5 CONTENT AI EVALUATION CHALLENGE	**10**
5.1 Introduction and Objective	10
5.2 Content Generation Using an NLP Tool	10
5.3 Evaluating Clarity and Tone	11
5.4 Evaluating Body-Language Cues in Multimedia	11
5.5 Findings, Ethics and Conclusion	11
5.6 Overall Conclusion and Future Scope	12
5.7 References	13

MODULE 2
BIG DATA PROCESS MAPPING

Question (Module 2): *Big Data Process Mapping: Groups select a real-world big data system (like Google Maps, Amazon recommendations, or smart city sensors) and map out the entire data flow: data sources, storage, processing, and output.*

2.1 Introduction and Problem Statement

Big data refers to information thatis too large, too fast, or too varied to be captured, stored, and analysed with traditional single-machine databases. It is usually described through the five Vs: volume, the quantity of records; velocity, the speed of arrival; variety, the mixture of tables, logs, images, and text; veracity, the trustworthiness of noisy data; and value, the insight that justifies the cost. Mapping a big data process means tracing data from the moment it is generated to the moment it influences a decision shown to a user.

The group selected Google Maps as its case study and organised the data flow under four headings: data sources, storage, processing, and output. It combines slowly changing planet-scale map data with live traffic data that changes every few seconds. The map presented here is a simplified, educational model built from publicly available descriptions; the internal architecture of a commercial product is proprietary, so the stages should be read as a representative pipeline rather than an official blueprint.

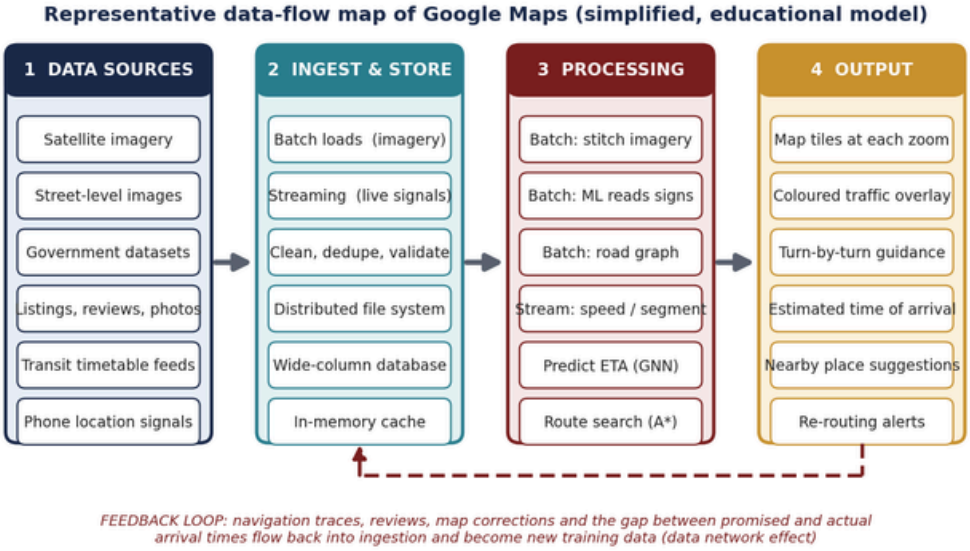


Figure 2.1: *End-to-end data-flow map of the case study, from data sources to user-facing output, with the feedback loop that returns every output to the system as new input.*

2.2 Data Sources

The first layer contains the places where data originates. The base map is assembled from satellite and aerial imagery, from street-level images captured by camera-equipped vehicles, and from authoritative datasets supplied by governments and mapping agencies, such as road networks, boundaries, and address records. Business and place details come from owner-submitted listings and from user contributions including reviews, photographs, and corrections, while public transport timetables arrive as structured feeds from transit agencies.

The most dynamic source is the stream of location signals from the phones of travellers who have chosen to share them. Each signal holds a position, a timestamp, and a speed estimate, and when millions of such records are aggregated and anonymised they reveal how fast traffic is moving on every road segment at that moment. Together these sources show the variety dimension of big data: images, geometries, tables, text, and telemetry at once.

2.3 Data Ingestion and Storage

The ingestion stage must receive data reliably however it arrives. Slowly changing data such as imagery is loaded in large batches, while live location and incident data is received as continuous streams through messaging and stream-processing services that absorb bursts of traffic, remove duplicates, and attach consistent timestamps. Records are also validated here: impossible speeds are discarded, coordinates are snapped to the nearest plausible road, and personal identifiers are removed. The storage layer then keeps each kind of data in the form that suits how it will be read. Imagery tiles and photographs are kept in distributed file systems that spread every file across many machines and replicate it against hardware failure. Structured records such as places and road segments are kept in distributed wide-column or globally consistent databases that scale horizontally by adding machines, and hot data such as the current speed of a busy highway is held in memory caches for millisecond access. Publicly documented Google technologies such as the Google File System, Colossus, Bigtable, and Spanner are examples of the infrastructure involved.

2.4 Processing Layer

Processing turns raw data into knowledge and runs in two cooperating modes. Batch processing periodically scans the full stored data to build and refresh the base map: imagery is stitched and aligned, machine-learning models read signboards, street names, and house numbers from photographs, and the road network is converted into a weighted graph in which every junction is a node and every road segment is an edge carrying length, speed limit, and turn restrictions.

Stream processing runs continuously over the live data. Incoming signals are grouped by road segment and short time window, averaged, and compared with the historical pattern for the same segment at the same hour, which produces a current speed and congestion level for every active road. Predictive models then estimate how traffic will change over the next minutes and hours; Google has publicly described using graph neural networks, which treat the road network as a graph, to improve the accuracy of arrival-time estimates. When a user requests directions, a shortest-path algorithm of the Dijkstra or A-star family runs over the graph with the predicted travel times as edge weights and returns candidate routes ranked by duration.

Figure 2.2 shows the routing step on a toy network whose edges carry predicted travel times in minutes. The congested segment B-D is expensive, so Dijkstra's algorithm returns the route A-C-E-F-G (19 minutes).

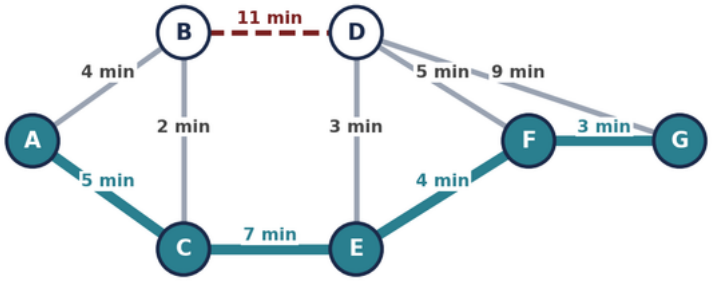


Figure 2.2: *Dijkstra search on a weighted road graph (illustrative values); the teal route avoids the congested edge.*

2.5 Output Layer and Feedback Loop

The output layer is everythingthe user sees and hears: map tiles at the correct zoom level, the coloured traffic overlay, turn-by-turn guidance, the estimated time of arrival, nearby place suggestions, and re-routing alerts when conditions change. These outputs reach phones, cars, and third-party websites through application programming interfaces and must appear in well under a second, so precomputed tiles and caches are used wherever possible.

The process map is a loop rather than a straight line. Once a user starts navigating, their phone becomes a new data source, reporting position and speed back to the ingestion stage, while reviews and map corrections flow into the place database and the gap between promised and actual arrival times becomes training signal for the prediction models. Every output therefore becomes an input, a property often called the data network effect. The main challenges noted were location privacy, noisy GPS readings, and uneven accuracy where there are fewer contributors.

2.6 The Five Vs Applied to Google Maps

Table2.1:ThefiveVs of Section 2.1 mapped to the case study

V	Meaning	Where it appears in the case study
Volume	Quantity of records	Planet-scale imagery, road geometry, place records and photographs
Velocity	Speed of arrival	Continuous location signals and incident reports; speeds refreshed every few seconds
Variety	Mixture of formats	Images, geometries, tables, review text, transit feeds and telemetry
Veracity	Trustworthiness of noisy data	Noisy GPS readings; impossible speeds discarded, coordinates snapped to roads
Value	Insight that justifies the cost	Accurate ETA, fast routes, up-to-date places and live re-routing

MODULE 3
BUILD A SIMPLE ML PROCESS FLOW

Question (Module 3): *Build a Simple ML Process Flow: Groups create a complete flowchart for a machine learning project, covering data collection, feature extraction, algorithm selection, training, testing, and evaluation.*

3.1 Introduction and Problem Definition

Machine learning builds programs that improveat a task by learning patterns from data instead of following hand-written rules. A project is therefore a disciplined sequence of stages, each producing an output that feeds the next and each able to send the team back to an earlier stage when results disappoint. The group's task was to draw this sequence as a flowchart covering data collection, feature extraction, algorithm selection, training, testing, and evaluation, and to explain every symbol in it.

To keep the chart concrete, a familiar supervised problem was used as a running example: predicting whether a telecom subscriber will cancel the service next month, known as customer churn prediction. The target is a yes-or-no label, so this is binary classification. The flowchart starts with a terminator symbol and a process box that defines the problem and success criteria, and then uses rectangles for processing steps, parallelograms for data input and output, diamonds for decisions, and arrows for the direction of flow.

3.2 Data Collection and Preparation

The first working stage is data collection. For the churn example the team gathers billing records, call and data-usage logs, support tickets, and contract details, and joins them on a customer identifier so that each row describes one subscriber and carries a label showing whether that subscriber left. At this point the team asks whether the data is large enough, representative of future customers, and legally permitted for this use.

Raw data is rarely ready for learning, so the next box is cleaning and preprocessing. Missing values are removed or filled with the median, the most frequent category, or a predicted value; duplicates are dropped; impossible entries such as negative call durations are corrected; and outliers are examined rather than deleted blindly. Categorical fields such as contract type are converted to numbers by one-hot encoding, and numeric fields are standardised so that columns with large units do not dominate.

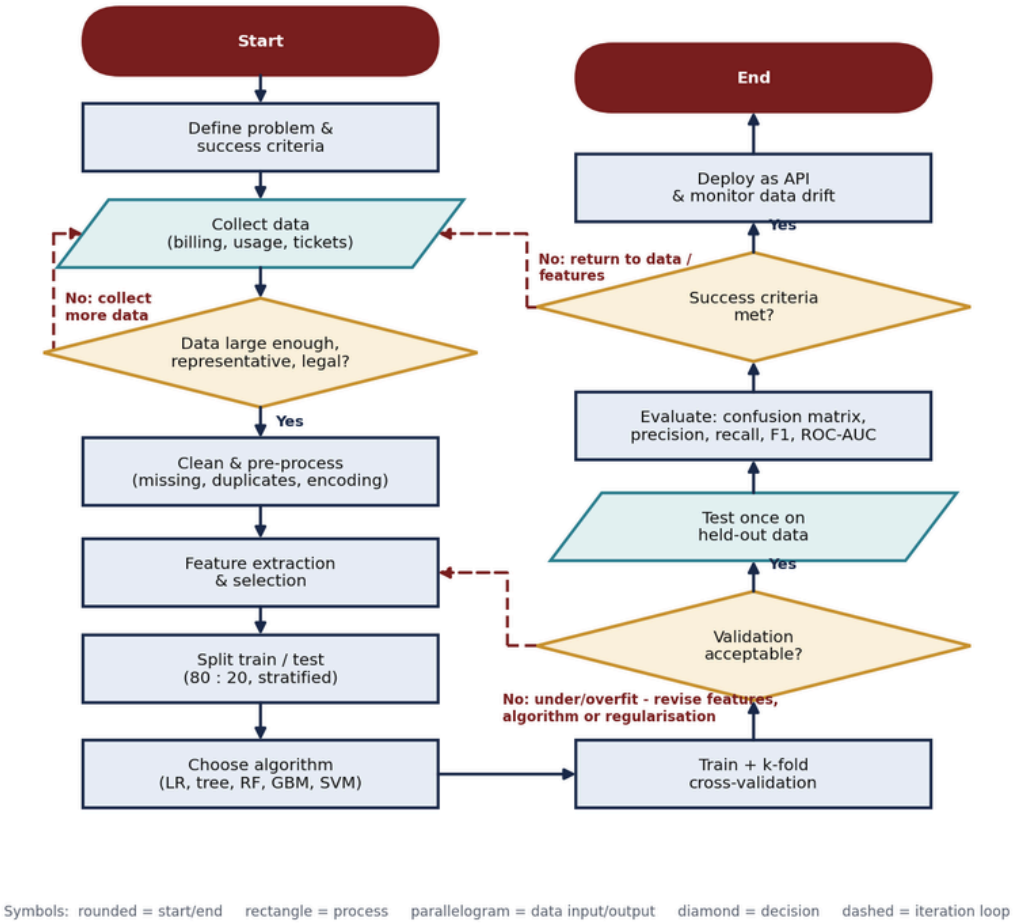


Figure 3.1: *Complete machine learning process flowchart for the churn-prediction example. Dashed arrows are the iteration loops back to earlier stages.*

3.3 Feature Extraction and Algorithm Selection

Feature extraction derives meaningful inputs from the cleaned data, because raw columns are seldom the best predictors. The team engineers features such as the average bill over six months, the change in data usage since the previous quarter, the number of complaints in ninety days, the length of the relationship, and the time left on the contract. A feature selection box then removes correlated or uninformative columns using correlation analysis, model-based importance scores, or principal component analysis, which shortens training and lowers the risk of overfitting. The data is finally split, with class proportions preserved, into a training portion of about eighty per cent and a held-out test portion that stays untouched until the end; scaling statistics are computed on the training portion only, so that no information leaks from the test data.

Algorithm selection is drawn as a decision, because the choice depends on the type and size of the data, the need for interpretability, and the available computing power. For tabular binary classification the candidates are logistic regression, a fast and interpretable baseline; decision trees, which give readable rules; random forests, which average many trees to reduce variance; gradient-boosted trees, which often achieve the best accuracy on structured data; and support vector machines or small neural networks for strongly non-linear patterns. Table 3.1 summarises the trade-offs.

Table 3.1: Candidate algorithms for tabular binary classification

Algorithm	Main strength	Main limitation
Logistic regression	Fast, interpretable baseline	Limited to mostly linear patterns
Decision tree	Readable if-then rules	Unstable; easily overfits
Random forest Gradient-	Averages many trees, lowers variance	Slower; harder to explain
boosted trees SVM / small	Often best accuracy on structured data	Needs careful tuning
neural network	Captures strongly non-linear patterns	Less interpretable; scaling needed

3.4 Training and Validation

In training,the chosen algorithms is fitted to the training data: it searches for the parameters, such as regression coefficients or tree split points, that minimise a loss function measuring the gap between predictions and true labels. Hyperparameters such as tree depth, the number of trees, or the learning rate are set by the designer, and to choose them without touching the test set the team uses k-fold cross-validation, in which the training data is divided into k parts and the model is trained k times, each time validating on a different part.

The flowchart contains a loop at this point. If both training and validation scores are low, the model is underfitting, and the team returns to feature engineering or picks a more expressive algorithm. If the training score is high but the validation score is much lower, the model is overfitting, and the team adds regularisation, simplifies the model, collects more data, or removes features. Only when validation performance is acceptable does the arrow leave the loop and continue to the testing stage. Figure 3.2(a) shows the typical pattern: training error keeps falling as the model grows more complex, while validation error falls and then rises again.

3.5 Testing, Evaluation and Deployment

Testingis the single, final measurement of the chosen model on the held-out data, which simulates the unseen customers it will meet in production. Predictions are summarised in a confusion matrix of true positives, false positives, true negatives, and false negatives, from which the team calculates accuracy, precision, recall, the F1-score, and the area under the ROC curve. Accuracy alone is misleading for churn, because a model that predicts that nobody leaves would still be ninety per cent accurate if only ten per cent of customers churn. Recall, the share of real churners caught, and precision, the share of flagged customers who would really have left, are more informative, and the balance between them is set by comparing the cost of a missed churner with the cost of a wasted retention offer.

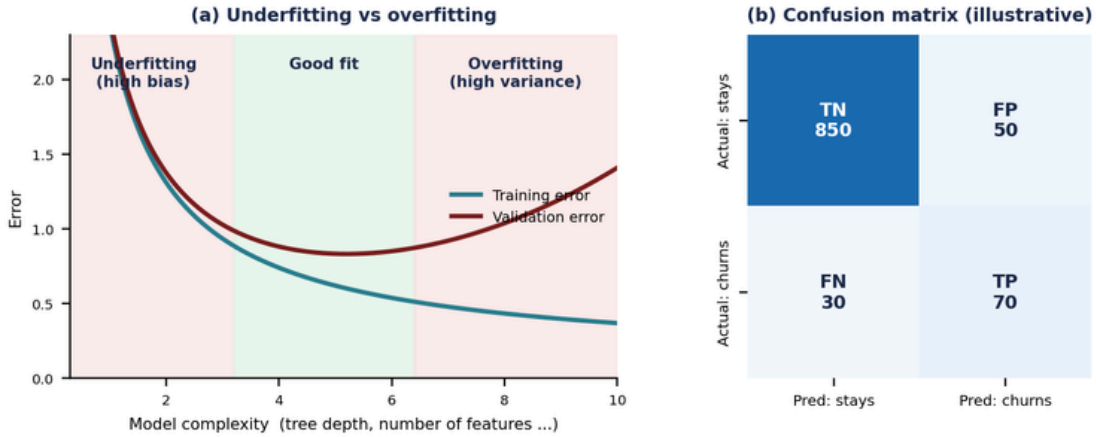


Figure 3.2: *(a) Underfitting, good fit and overfitting as model complexity grows. (b) A confusion matrix for 1,000 hypothetical customers, 10 per cent of whom churn (illustrative numbers).*

Table 3.2: Metrics computed from the matrix in Figure 3.2(b)

Metric	Formula	Value	Reading
Accuracy	(TP + TN) / all	92.0 %	Looks high, but a model that predicts 'nobody leaves' scores 90 %
Precision	TP / (TP + FP)	58.3 %	Of 120 customers flagged, 70 really would have left
Recall	TP / (TP + FN)	70.0 %	Of 100 real churners, 70 were caught (the 'nobody leaves' model catches 0)
F1-score	2PR / (P + R)	63.6 %	Single balance of precision and recall

Evaluation also inspects feature importance and error cases, and compares the result with the baseline and the success criteria written at the start. The final decision diamond asks whether those criteria are met. If they are, the model is packaged behind an application programming interface for deployment and monitored for data drift so that it can be retrained when customer behaviour changes; if not, an arrow returns to data collection or feature extraction, showing that machine learning is an iterative cycle rather than a one-way pipeline.

MODULE 4
BACKPROPAGATION WALKTHROUGH

Question (Module 4): *Backpropagation Walkthrough: Together, step through a simple neural network's forward and backward pass calculations, showing gradient flow and weight updates.*

4.1 Introduction and Network Setup

Backpropagation isthe algorithm that makes the training of neural networks practical. A network is a chain of simple mathematical operations, and training means adjusting its weights so that its output moves closer to the desired answer. Backpropagation first runs an input forward through the network to obtain a prediction, then measures the error, and finally applies the chain rule of calculus backwards from the output to every weight, revealing how much each weight contributed to the error and in which direction it should move. The group used a deliberately small network so that every number could be checked by hand.

The network has two input neurons, one hidden layer of two neurons, and one output neuron, and every hidden and output neuron uses the sigmoid activation, sigmoid(z) = 1 / (1 + e^{-z}), whose derivative equals sigmoid(z) multiplied by (1 - sigmoid(z)). The inputs are x1 = 0.5 and x2 = 0.8 and the target is t = 1. The weights into the first hidden neuron are w1 = 0.4 and w2 = 0.3, those into the second are w3 = 0.2 and w4 = 0.6, and the hidden bias is b1 = 0.1. The output weights are w5 = 0.5 and w6 = 0.7, the output bias is b2 = 0.2, the loss is E = 0.5 × (t - o)^2 for output o, and the learning rate is 0.5.

4.2 Forward Pass and Loss

In the forward pass each neuron computes a weighted sum of its inputs plus its bias, called the net input z, and applies the activation to obtain its output. For the first hidden neuron z1 = (0.4 × 0.5) + (0.3 × 0.8) + 0.1 = 0.54, so h1 = sigmoid(0.54) = 0.6318. For the second, z2 = (0.2 × 0.5) + (0.6 × 0.8) + 0.1 = 0.68, so h2 = sigmoid(0.68) = 0.6637. These outputs feed the output neuron, whose net input is z3 = (0.5 × 0.6318) + (0.7 × 0.6637) + 0.2 = 0.9805, giving o = sigmoid(0.9805) = 0.7272. The network predicts 0.7272 when the desired answer is 1.

The loss is E = 0.5 × (1 - 0.7272)^2 = 0.0372, where the factor of one half is included only so that it cancels with the exponent during differentiation. To reduce this loss the network must know, for each of its eight adjustable parameters, whether increasing it would raise or lower the error. That information is the gradient, and calculating it is the purpose of the backward pass.

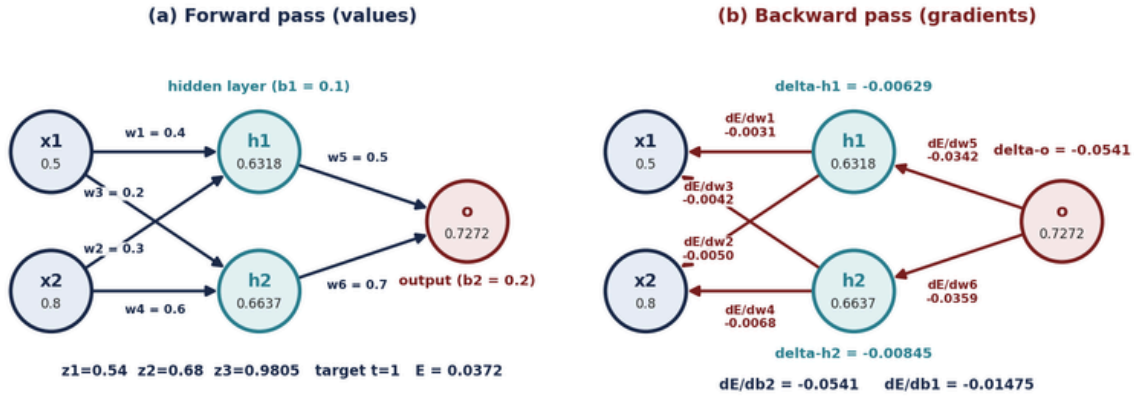


Figure 4.1: *The 2-2-1 network used in the walkthrough. (a) Values computed in the forward pass; (b) the error signals and gradients computed in the backward pass, which retrace the same connections in reverse.*

4.3 Backward Pass and Gradient Flow

The backward pass starts at the output and moves toward the input, reusing earlier results at every step. First, dE/do = o - t = 0.7272 - 1 = -0.2728, and this is multiplied by the sigmoid slope at the output, o(1 - o) = 0.7272 × 0.2728 = 0.1984, giving the output error signal delta-o = -0.0541. In chain-rule form, dE/dw5 = (dE/do) × (do/dz3) × (dz3/dw5), and the last factor is simply h1. Each output-layer gradient is therefore this signal multiplied by the value the weight carried forward: dE/dw5 = -0.0541 × 0.6318 = -0.0342, dE/dw6 = -0.0541 × 0.6637 = -0.0359, and dE/db2 = -0.0541, since a bias is multiplied by the constant one. The negative signs show that raising these weights would lower the loss.

The error signal then flows back into the hidden layer, which is the heart of backpropagation. The share belonging to each hidden neuron depends on the connecting weight, so the first receives -0.0541 × 0.5 = -0.0271 and the second -0.0541 × 0.7 = -0.0379. Each multiplies its incoming signal by its own sigmoid slope, giving delta-h1 = -0.0271 × 0.6318 × (1 - 0.6318) = -0.00629 and delta-h2 = -0.0379 × 0.6637 × (1 - 0.6637) = -0.00845. The first-layer gradients are these deltas times the inputs: dE/dw1 = -0.00315, dE/dw2 = -0.00504, dE/dw3 = -0.00423, and dE/dw4 = -0.00676, while the shared hidden bias receives both signals, dE/db1 = -0.00629 + (-0.00845) = -0.01475.

4.4 Weight Update and Verification

Gradient descent now moves every weight a small step against its gradient: new weight = old weight minus learning rate × gradient. Because every gradient is negative, every parameter increases slightly, which pushes the prediction upward toward the target. The output layer becomes w5 = 0.5 - 0.5 × (-0.0342) = 0.5171, w6 = 0.7180, and b2 = 0.2271, and the first layer becomes w1 = 0.4016, w2 = 0.3025, w3 = 0.2021, w4 = 0.6034, and b1 = 0.1074. All eight values are collected in Table 4.1.

Table 4.1: Gradients and updated parameters after one step

Parameter	Old value	Gradient dE/d(parameter)	New value = old - 0.5 x gradient
w1	0.4	-0.00315	0.4016
w2	0.3	-0.00504	0.3025
w3	0.2	-0.00423	0.2021
w4	0.6	-0.00676	0.6034
w5	0.5	-0.03419	0.5171
w6	0.7	-0.03592	0.7180
b1	0.1	-0.01475	0.1074
b2	0.2	-0.05411	0.2271

A second forward pass with the updated weights confirms that learning has occurred: the hidden outputs become h1 = 0.6342 and h2 = 0.6662, the prediction rises from 0.7272 to 0.7376, and the loss falls from 0.0372 to 0.0344 (Table 4.2). One step gives a small improvement, as expected with a modest learning rate, and repeating the cycle of forward pass, loss, backward pass, and update over many iterations and many examples steadily drives the loss toward its minimum.

Table 4.2: Verification with a second forward pass

Quantity	Before update	After update
Hidden output h1	0.6318	0.6342
Hidden output h2	0.6637	0.6662
Network output o	0.7272	0.7376
Loss E	0.0372	0.0344

4.5 Observations

Three lessons emerged. First, the hidden-layer deltas are roughly an order of magnitude smaller than the output delta, because every step backwards multiplies the signal by a sigmoid slope that can never exceed 0.25; in deep networks this repeated shrinking causes the vanishing gradient problem, one reason modern networks prefer ReLU activations, careful initialisation, normalisation layers, and residual connections. Second, the learning rate sets the step size: too small and training is slow, too large and the updates overshoot the minimum and the loss may oscillate or diverge. Third, backpropagation is efficient because each error signal is computed once and reused, so the backward pass costs about the same as the forward pass, which is what allows networks with billions of parameters to be trained. In practice, examples are processed in mini-batches whose gradients are averaged before each update, and optimisers such as momentum and Adam refine the plain gradient-descent step used here. Frameworks such as PyTorch and TensorFlow automate the process through automatic differentiation, but the manual walkthrough remains valuable for debugging and for understanding new architectures.

Figure 4.2 extends the experiment: repeating the same update for 300 iterations on this single example drives the loss from 0.0372 to about 0.0008, and the sigmoid plot shows why the signal shrinks at every layer.

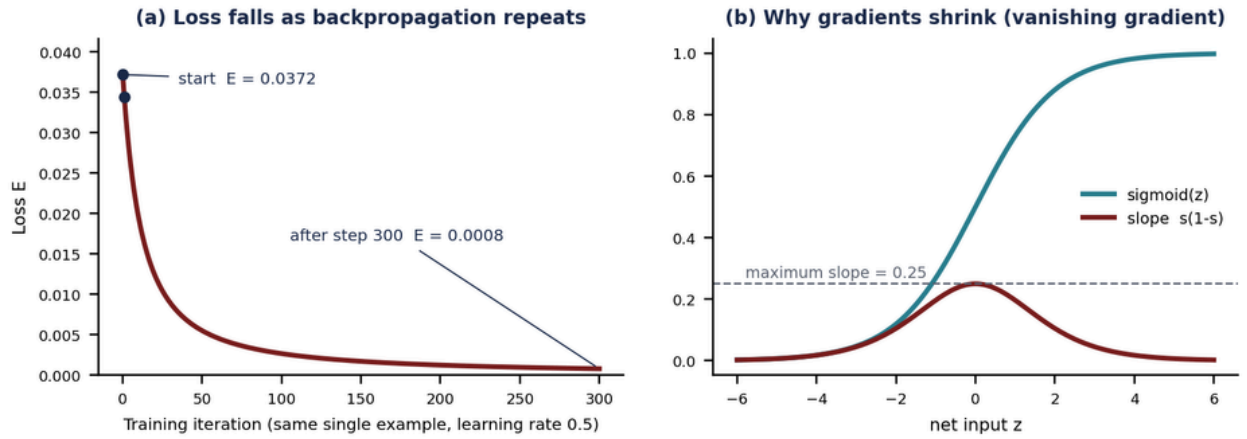


Figure 4.2: *(a) Loss against training iteration for the walkthrough network (computed by repeating the update above). (b) The sigmoid function and its slope, which never exceeds 0.25.*

Table 4.3 repeats the experiment with different learning rates. A small rate such as 0.1 is still slow after 20 iterations, while larger rates reach a low loss sooner; on this tiny, smooth problem even a rate of 2.0 stays stable, but on deeper networks such large steps are what cause the oscillation or divergence described above.

Table 4.3: Effect of the learning rate (same network and example, computed by repetition)

Learning rate	Loss after 20 iterations	Loss after 300 iterations
0.1	0.0280	0.0046
0.5	0.0125	0.0008
1.0	0.0067	0.0004
2.0	0.0033	0.0002

MODULE 5
CONTENT AI EVALUATION CHALLENGE

Question (Module 5): *Content AI Evaluation Challenge: Groups generate content using an NLP tool, then evaluate clarity, tone, and body-language cues for multimedia formats.*

5.1 Introduction and Objective

Natural language processing, or NLP,is the branch of artificial intelligence that enables computers to read, interpret, and generate human language. Modern generative tools are built on transformer-based large language models, trained on vast text collections to predict the next word, which lets them draft essays, emails, and scripts on request. Because such tools produce fluent text instantly, the real challenge is no longer producing content but judging whether it is good enough to use. This challenge was designed around that skill: the group generates content with an NLP tool and then evaluates it on clarity, tone, and, for multimedia formats, the body-language cues that accompany the words.

The objectives were to practise writing effective prompts, to build a transparent rubric that several reviewers can apply consistently, and to understand why the quality of a text and of its delivery must be assessed separately when the same message becomes a video or an avatar speech.

5.2 Content Generation Using an NLP Tool

The task adopted was a two-hundred-and-fifty-word script in which a university introduces a new artificial intelligence laboratory to prospective students, prepared as a web article, a spoken narration, and a short video presented by a digital avatar. The language model received a structured prompt stating role, audience, purpose, length, tone, and constraints, for example: act as an admissions officer, address first-year applicants, use a friendly but professional voice, avoid jargon, and end with a call to action. Variants were produced by changing one element at a time, such as tone or audience, so that the effect of each change could be observed, and every draft was saved with the prompt that created it.

Table 5.1: Elements of the structured prompt used in the challenge

Element	Setting used in the task
Role	Act as an admissions officer
Audience	First-year applicants
Purpose	Introduce the new artificial intelligence laboratory
Length	Two hundred and fifty words
Tone	Friendly but professional
Constraints	Avoid jargon; end with a call to action

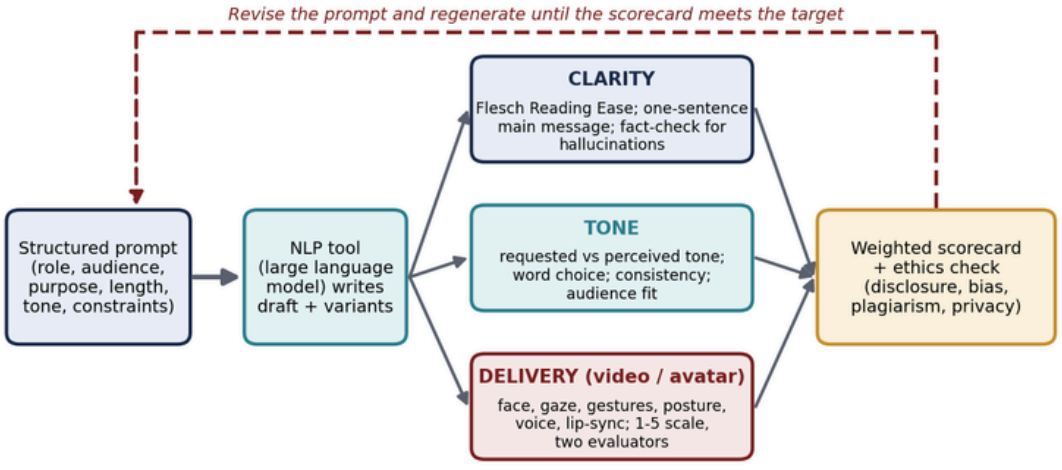


Figure 5.1: *The generate-and-evaluate workflow: a structured prompt produces drafts, three evaluation lanes score them, and the scorecard decides whether the prompt must be revised.*

5.3 Evaluating Clarity and Tone

Clarity measures how easily the intended audience understands the message on first reading or hearing. Readability formulas such as the Flesch Reading Ease score estimate difficulty, and qualitative checks ask whether the main message can be stated in one sentence, whether each paragraph holds one idea, and whether the text is free of ambiguity and filler. Factual accuracy is verified separately against reliable sources, because generative models can produce confident but invented statements, known as hallucinations, which make an otherwise clear text misleading.

Tone is the attitude a text conveys, such as formal, friendly, persuasive, or empathetic. Evaluators compare the tone requested in the prompt with the tone readers actually perceive, examine word choice and sentence rhythm, and check consistency from start to finish. Sentiment classifiers give a rough indication, but human judgement remains the final authority, because sarcasm and cultural context are hard for software to detect. A tone that suits a social media caption can be unsuitable in an official letter, which is why tone must be judged against the audience and purpose.

5.4 Evaluating Body-Language Cues in Multimedia

When the same text is delivered through video,a presentation, oran avatar, meaning is carried by non-verbal signals as well as words, and an excellent script can be undermined by poor delivery. The rubric therefore examines facial expressions, which should match the emotional content of the words; gaze and eye contact, which should be directed at the audience or camera; gestures, which should emphasise key points and stay synchronised with speech rather than looking repetitive or mechanical; posture and head movement, which signal confidence; and vocal delivery, including pace, pauses, pitch, and emphasis. Lip synchronisation is also checked, since small mismatches create the uncanny valley effect, in which an almost-human figure appears unsettling.

Each cue is scored on a five-point scale with written descriptors, from one for absent or distracting to five for natural and fully supportive, and at least two evaluators score independently before differences are discussed or measured with an agreement statistic such as Cohen’s kappa. Because the meaning of gestures and eye contact varies between cultures, the scores are interpreted with the intended audience in mind. Table 5.2 shows the rubric in a form reviewers can apply directly.

Table 5.2: Body-language and delivery rubric (five-point scale)

Cue	1 - absent or distracting	3 - acceptable	5 - natural and fully supportive
Facial expression	Blank or contradicts the words	Matches the words some of the time	Matches the emotional content throughout
Gaze and eye contact	Looks away; no contact with audience	Occasional contact with camera	Directed at the audience or camera consistently
Gestures	Absent, or repetitive and mechanical	Present but loosely timed	Emphasise key points, synchronised with speech
Posture and head movement	Stiff, slouched or erratic Monotone; wrong pace	Neutral and stable	Signals confidence and engagement
Vocal delivery	Obvious mismatch (uncanny valley)	Clear but flat in places	Good pace, pauses, pitch and emphasis
Lip synchronisation		Minor lapses	Mouth movement matches the audio

5.5 Findings, Ethics and Conclusion

Combining the three dimensions gives a balanced scorecard in which clarity, tone, and delivery are reported separately and then weighted according to the purpose of the content. Generative tools are strongest at producing well-structured first drafts and adapting a message to different audiences, but they need human oversight for accuracy, originality, and cultural sensitivity. Ethical practice also forms part of the evaluation: AI assistance should be disclosed where appropriate, confidential data should never be pasted into public tools, outputs should be checked for bias and unintended plagiarism, and synthetic voices or faces must never be used to impersonate real people. Evaluating AI-generated content is thus a structured, repeatable process rather than a matter of impression.

Because the weighting depends on the purpose of the content, a web article leans on clarity while an avatar video leans on delivery. Table 5.3 gives one example of how a team might set the weights; the percentages are suggestions, not measured results.

Table 5.3: Example weighting of the scorecard by format (suggested values)

Format	Clarity	Tone	Delivery
Web article	55 %	45 %	not scored
Spoken narration	40 %	35 %	25 %
Avatar video	30 %	25 %	45 %

5.6 Overall Conclusion and Future Scope

The four modules of this group task follow one idea from the data upward. Module 2 showed how a big data system such as Google Maps moves data through sources, ingestion and storage, processing and output, and how its outputs return as new inputs. Module 3 turned the processing step into a disciplined machine learning workflow in which every decision diamond can send the team back to an earlier stage. Module 4 opened the neural network that sits inside many such models and verified by hand that a single backpropagation step lowers the loss from 0.0372 to 0.0344. Module 5 addressed the newest layer, generative AI, and argued that fluent output must still be judged for clarity, tone, delivery and ethics.

Several extensions are possible. The big data map could be repeated for a second system, such as a recommendation engine, and the two compared on the five Vs. The churn flowchart could be implemented with a public dataset and the metrics in Table 3.2 measured instead of illustrated. The backpropagation example could be reproduced in PyTorch to confirm the hand calculation with automatic differentiation, and the Module 5 rubric could be applied by several reviewers to real drafts so that agreement can be reported with Cohen’s kappa.

5.7 References

1. Marz, N., & Warren, J., “Big Data: Principles and Best Practices of Scalable Real-Time Data Systems”.
2. Géron, A., “Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow”, O’Reilly Media.
3. Rumelhart, Hinton, & Williams, “Learning representations by back-propagating errors”, Nature, 1986.
4. Goodfellow, I., Bengio, Y., & Courville, A., “Deep Learning”, MIT Press.
5. Vaswani, A., et al., “Attention Is All You Need”, NeurIPS, 2017.
6. V. Mayer-Schönberger and K. Cukier, Big Data: A Revolution That Will Transform How We Live, Work, and Think. Houghton Mifflin Harcourt, 2013.
7. A. Géron, Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 3rd ed. O’Reilly Media, 2022.
8. I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. MIT Press, 2016. Available: https://www.deeplearningbook.org/
9. Apache Software Foundation, “Apache Hadoop.” Available: https://hadoop.apache.org/
10. Scikit-learn Developers, “Scikit-learn: Machine Learning in Python.” Available: https://scikit-learn.org/stable/
11. Google, “TensorFlow: An End-to-End Machine Learning Platform.” Available: https://www.tensorflow.org/
12. Google Scholar, “Research Articles on Big Data, Machine Learning, Neural Networks, and Generative AI.” Available: https://scholar.google.com/
13. OpenAI, “Artificial Intelligence and Generative AI Resources.” Available: https://openai.com/
14. The Elements of Statistical Learning — Trevor Hastie, Robert Tibshirani, and Jerome Friedman, 2009.
15. Data Mining and Analysis: Fundamental Concepts and Algorithms — Mohammed J. Zaki and Wagner Meira Jr., 2014.
16. Understanding Machine Learning: From Theory to Algorithms — Shai Shalev-Shwartz and Shai Ben-David, 2014.
17. Vaswani, A., et al. (2017). “Attention Is All You Need.” Advances in Neural Information Processing Systems.
18. Bommasani, R., et al. (2021). “On the Opportunities and Risks of Foundation Models.” Stanford University.
19. Bender, E. M., et al. (2021). “On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?” Proceedings of FAccT ’21.
20. Breiman, L. (2001). “Statistical Modeling: The Two Cultures.” Statistical Science, 16(3), 199–231.